

Logos Runtime vs Compile Time Costs

<https://gemini.google.com/app/450de56dc3a49313>

User prompt: Anything faster? I tried to hand roll swar (well an agent did) and it was about same. How's it so fast? What is swar vs simd?

Response: Anything Faster than Logos?

Yes, but usually **only under specific constraints**:

1. **Explicit Vectorization (AVX2/AVX-512/NEON)**: Tools using raw vector intrinsics (like `memchr` or custom SIMD-based lexers) can sweep through text using 128-, 256-, or 512-bit registers. They are faster for finding simple byte boundaries or delimiters (e.g., quotes, brackets, newlines) at 10+ GB/s, but struggle with complex state-heavy regular expression logic.
2. **Aho-Corasick / Hyperscan**: If you are only searching for fixed sets of static strings or patterns rather than full regular expressions with custom token enum variants, specialized state engines like Hyperscan/Vectorscan can beat Logos.
3. **Compile-time specialized lookup tables**: Hand-written state machines tuned precisely for your exact grammar (like a specialized JSON/HTTP parser using `pcmpestr` or AVX shuffle tables) can beat general-purpose regex-based lexers.

Why Is Logos So Fast?

Logos isn't fast because of magic; it's fast because it avoids CPU branch mispredictions and optimizes L1/L2 cache behavior:

1. **DFA Minimization**: Logos converts all your token rules into a single global **Deterministic Finite Automaton (DFA)** and minimizes the states (Hopcroft's algorithm). There is no backtracking, no NFA simulation, and no stack frames.
2. **Pre-computed Lookup Tables**: Transitions are generated as direct arrays or dense `match` jump tables. The CPU branch predictor quickly learns the loop jump targets.
3. **Fast Path Batching**: Logos automatically groups common prefixes and attempts word-sized reads (like 4 or 8 bytes at a time) when patterns permit, avoiding byte-by-byte iteration where possible.
4. **Zero-Copy / Zero-Allocation**: It works strictly on `&str` or `&[u8]` slices without allocating heap memory for tokens or spans.

What is SWAR vs. SIMD?

Both **SWAR** and **SIMD** are techniques for processing multiple data elements in parallel using a single instruction (**Single Instruction, Multiple Data**), but they operate at different architectural levels.

Feature	SWAR (SIMD Within A Register)	Hardware SIMD (SSE/AVX/NEON)
Hardware Used	General-purpose scalar registers (e.g., standard <code>u64 / usize</code>).	Specialized vector registers (e.g., 128-bit <code>xmm</code> , 256-bit <code>ymm</code>).
Portability	100% portable. Runs on any architecture with basic bitwise math.	Architecture-dependent (x86_64 vs. ARM) unless using abstractions like <code>std::simd</code> .
Implementation	Bit manipulation tricks (masks, shifts, bitwise AND/OR, multiplication).	Dedicated CPU instructions (<code>_mm256_cmpeq_epi8</code> , <code>paddb</code> , etc.).
Throughput	Processes up to 8 bytes per iteration on 64-bit CPUs.	Processes 16, 32, or 64 bytes per iteration.
Cross-Lane Overhead	Requires manual bit-masking to prevent carries from bleeding into adjacent bytes.	Hardware automatically handles lane isolation and arithmetic carries.

How SWAR Works (Conceptual)

Suppose you want to check if any byte in an 8-byte chunk (`u64`) contains an ASCII space (`0x20`) or a null byte (`0x00`).

Instead of writing a `for` loop over 8 bytes:

- **Scalar approach:** 8 loop iterations, 8 branches.
- **SWAR approach:** XOR the entire `u64` with `0x2020202020202020`, then run a bit-magic mask (like `(x - 0x0101...) & ~x & 0x8080...)` to check for zero bytes across all 8 slots in **one CPU operation**.

Because SWAR operates on 64-bit standard registers, it often matches simple SIMD performance for light scanning tasks due to zero setup overhead and zero register cross-talk costs.

User prompt: So swar isn't machine instructions or true simd. It's just data design to do branchless lookups over u64?

Response: It **is** composed of real machine instructions, but **not hardware-level SIMD instructions**.

You hit the nail on the head regarding the strategy: SWAR is an algorithmic trick that repurposes standard scalar 64-bit integer instructions (**ADD**, **SUB**, **AND**, **XOR**, shifts) to execute "SIMD-style" parallel logic across multiple smaller sub-words (like eight 8-bit bytes) packed into a single standard CPU register.

Key Distinctions

1. Machine Instructions

- **SWAR:** Uses standard scalar CPU instructions that operate on general-purpose registers (e.g., `rax / rdi` on x86, `x0 / x1` on ARM).
- **True SIMD:** Uses specialized CPU vector instructions (e.g., AVX-256 `vpcmpeqb` or ARM NEON `vceq_u8`) that operate on dedicated 128-, 256-, or 512-bit vector registers.

2. The Mechanics

SWAR stands for **SIMD Within A Register**. When you load 8 bytes into a `u64`, the CPU views it as one large scalar number. To treat those 8 bytes as separate parallel channels without vector hardware, SWAR uses **bit arithmetic tricks** to prevent math operations (like addition or subtraction) from leaking "carries" across byte boundaries.

A Concrete Example: Checking 8 Bytes for Nulls (`0x00`)

Without SIMD hardware, how do you check 8 bytes in parallel with zero branches?

You use the classic **Alan Mycroft SWAR algorithm** (which `memchr` and string libraries historically used on non-SIMD platforms):

```
#[inline]
pub fn has_zero_byte(x: u64) -> bool {
    // Subtract 1 from every byte; if a byte was 0x00, it wraps around to 0xFF (setting 0x80 bit)
    // Mask off the non-top bits, then check if high bits survived
    const SWAR_L0: u64 = 0x0101_0101_0101_0101;
    const SWAR_HI: u64 = 0x8080_8080_8080_8080;
```

```
((x.wrapping_sub(SWAR_LO)) & !x & SWAR_HI) != 0  
}
```

What happens at the CPU level here?

1. **No Branches (if statements):** The CPU runs 4 arithmetic instructions in a straight line.
2. **Parallel Evaluation:** All 8 byte positions are calculated at the exact same time inside the standard ALU.
3. **Outcome:** A single boolean output indicating whether any of the 8 bytes matched your target condition.

Why use SWAR over True SIMD?

If hardware SIMD exists, why bother with SWAR?

- **Zero Startup / Warmup Cost:** True SIMD often has alignment requirements, scalar-to-vector register transfer overhead, or clock-frequency throttling (on older x86 CPUs using AVX-512). SWAR operates directly on standard registers already sitting in the L1 pipeline.
- **Portable & Guaranteed:** It works on every 64-bit target under the sun—microcontrollers, WASM, old ARM, or modern x86—without needing `#![feature(portable_simd)]` or architecture target feature detection at runtime.
- **Effective for Small Workloads:** For short tokens or strings (e.g., under 16-32 bytes), SWAR's zero setup cost often beats true SIMD because vector register loading overhead negates the throughput gain.

User prompt: What's the difference in rust `b'string'` and `&str`? Does using the byte prefix just mean it never had to be interpreted at `ift8` and gives you shorthand for matching `utf8`? Ie. Logos is a lexer that can take `u8`. If that `u8` is `utf8`, is there any difference?

Response: You've hit on a fundamental distinction in Rust's type system and safety guarantees.

The short answer: **`b"string"` produces a byte slice (`&[u8; N]/&[u8]`), while `"string"` produces a UTF-8 string slice (`&str`).**

Using `b"..."` does **not** perform any UTF-8 validation at runtime. It is purely a compile-time convenience for writing arrays of ASCII/raw bytes.

Key Differences

Feature	String Literal ("foo")	Byte String Literal (b"foo")
Type	<code>&'static str</code>	<code>&'static [u8; 3]</code> (coerces to <code>&'static [u8]</code>)
Guarantees	Guaranteed valid UTF-8 at compile/runtime.	No UTF-8 guarantees. Just a raw array of <code>u8</code> numbers.
Allowed Characters	Full Unicode (e.g., "🦀 hello").	Strictly ASCII characters or <code>\xHH</code> byte escapes.
Memory Layout	Sequence of bytes in <code>.rodata</code> .	Exact same sequence of bytes in <code>.rodata</code> .

Does `b" . . . "` just skip UTF-8 interpretation?

Yes.

When you write a normal string `"hello"`, the Rust compiler verifies at compile time that every byte in that string forms valid UTF-8. At runtime, a `&str` is physically just a pointer and a length (`Fat Pointer`), with the explicit type-level guarantee that the memory behind it is valid UTF-8.

When you write `b"hello"`, the compiler simply takes the ASCII character codes (e.g., `'h' → 0x68`) and emits a raw byte array (`[u8; 5]`). It never checks or enforces UTF-8 invariants.

In Logos: Is there any difference if the `u8` input happens to be valid UTF-8?

At runtime: No performance difference.

In type safety & API mechanics: Yes, significant differences.

Because valid UTF-8 *is* just a sequence of `u8` bytes, Logos treats both `&str` and `&[u8]` almost identically under the hood. In fact, Logos implements its `Source` trait for both `&str` and `&[u8]`.

However, choosing between `u8` lexing vs `&str` lexing changes three key behaviors:

1. Token Slice Output (`lexer.slice()`)

- If you pass `&str` into Logos, `lexer.slice()` returns `&str`. You can directly pass tokens into functions that expect string slices without parsing/validation.
- If you pass `&[u8]` into Logos, `lexer.slice()` returns `&[u8]`. If you want to use that token as a string later, you must call `std::str::from_utf8(slice)` (or `from_utf8_unchecked`).

2. Byte Boundaries vs Character Boundaries

Logos indices and spans are **always byte offsets** (`Range<usize>`), regardless of input type.

- On `&str`, slicing at non-ASCII boundaries can panic if you slice mid-code-point manually.
- On `&[u8]`, slicing anywhere is always safe at the Rust memory level (though you might split a multi-byte UTF-8 sequence in half).

3. Matching Non-ASCII Bytes

- In `&str` Logos matching, matching something like `ä` matches its two-byte UTF-8 representation (`0xC3 0xA4`).
- In `&[u8]` Logos matching, you can match arbitrary binary data or non-UTF-8 protocols (e.g., `\xFF`, network packets, compressed streams) that would fail `&str` validation before the lexer even sees them.

Summary Checklist

- **Use `&str` / `"literal"`** when your input is text, and you want your lexer output tokens to automatically be valid `&str` references without converting them down the line.
- **Use `&[u8]` / `b"literal"`** when lexing raw binary data, network packets, or when you want to avoid paying for pre-validation of large inputs before feeding them to the lexer.

User prompt: Nom/winnow vs logos for creating a lexer? Let's say for usfm who has lots of new lines traditionally and a small vocab of lexemes (say like 10, markers, number, text, attribute, caller, etc;)

Response: For **USFM (Unified Standard Format Markers)**, **winnow** is almost certainly the better architectural fit, even if `logos` is technically faster at pure byte scanning.

While USFM looks like a simple text-with-markup format on the surface, its lexing rules are **highly context-dependent**. The way USFM treats text depends heavily on whether you are sitting inside standard verse text, an inline attribute block (`|x-foo="bar"`), a footnote (`\f + ... \f*`), or a nested marker (`\+w ... \+w*`).

The Fundamental Conflict

- **Logos** assumes a **lexing-first, context-free** model. It transforms regular expressions into a single global DFA state machine.
- **winnow** is a **parser combinator** designed for streaming or zero-copy string manipulation. It allows you to freely interleave matching tokens and parsing decisions in a single pass without a rigid, separate lexing phase.

Why Logos Struggles with USFM

Even though USFM has a small vocabulary of marker types, its tokenization rules switch based on state:

1. Contextual "Text" vs. Tokens

In USFM, almost anything that isn't a backslash marker (`\p`, `\v 1`, `\f`) or an end-marker (`\f*`) is arbitrary string content.

- In **Logos**, defining a catch-all `Text(&'a str)` token rule like `[^\\|*]+` easily creates ambiguity or overlapping token definitions with whitespace, numbers, or specific callers (+).
- Logos has modal sub-lexers (`#[logos(submode)]`), but switching states on backslashes and endmarkers quickly turns your token enum into a complex state graph.

2. Significance of Whitespace & Newlines

USFM is notoriously weird about whitespace.

- A space immediately following a marker (`\v 12`) is a **delimiter**.
- A space inside verse text is **data**.
- A newline before a paragraph marker (`\n\p`) signals a structural boundary.
- A newline after a verse marker can just be whitespace normalization.

In `logos`, handling whitespace dynamically based on the preceding token requires maintaining custom state in the lexer extra context. In `winnow`, you just write `space0` or `line_ending` exactly where the USFM spec expects it.

3. Inline Attributes (`\w word|lemma="grc123"\w*`)

USFM 3.0 character attributes require switching from normal text scanning to key-value string scanning. With `winnow`, when you hit `|`, you simply invoke your `attribute_parser` combinator, consume key-value pairs until you hit the closing marker, and return directly to normal text parsing.

Why Winnow Wins for USFM

With `winnow`, you can write a "lexer-less" or "integrated lexer" parser:

```
use winnow::prelude::*;
use winnow::token::{take_till, take_while};
use winnow::ascii::digit1;

// Matching a USFM marker like \v 12 or \p
fn usfm_marker<'s>(input: &mut &'s str) -> PResult<UsfmToken<'s>> {
    '\\'.parse_next(input)?;
    let name = take_while(1.., ('a'..'z', 'A'..'Z', '0'..'9', '+')).parse_next(input)?;

    // USFM markers are delimited by a space or newline
    let _ = opt(' ').parse_next(input)?;

    Ok(UsfmToken::Marker(name))
}

// Matching regular verse text up to the next marker or special character
fn usfm_text<'s>(input: &mut &'s str) -> PResult<&'s str> {
    take_till(1.., ['\\', '|', '*']).parse_next(input)
}
```

Advantages of Winnow here:

1. **Zero-Copy / Zero-Allocation:** `winnow` works natively on `&str` and slices. Your tokens and text chunks are just slice windows over the original USFM file.
2. **Modal Parsing is Natural:** You don't need a formal "lexer phase" followed by a "parser phase." You parse USFM structures directly from the raw `&str` using zero-cost combinators.
3. **Better Error Recovery:** When a translator leaves an unclosed inline style or malformed attribute in USFM, `winnow` allows you to fall back to treating the broken marker as plain text without cratering the entire DFA engine.

Trade-Off Summary

Feature	Logos	Winnow
Pure Byte Scanning Speed	Slightly faster (DFA jump tables)	Fast (Optimum slice bounds checking)
Context Switching	Hard (requires sub-lexers / state)	Trivial (parser combinators)
Handling Arbitrary "Text"	Tricky (regex overlap issues)	Trivial (<code>take_till</code> , <code>take_while</code>)
Whitespace Sensitivity	Cumbersome	Built-in (<code>space1</code> , <code>line_ending</code>)
Architecture	Lexer only (needs a separate parser)	Combined Lexer + Parser in one step

Recommendation

If USFM were a rigid programming language with strict lexical boundaries (like C or JSON), `logos` + a parser crate would be ideal. Because USFM is **document markup with contextual whitespace and arbitrary inline text spans**, go with `winnow`.

Sources

1. [Recommended Lexer for compilers dev in Rust? - Reddit](#)
 2. [maciejhirszt/logos: Create ridiculously fast Lexers - GitHub](#)
 3. [winnow - Rust - Docs.rs](#)
 4. [Syntax Notes — Unified Standard Format Markers 3.0.0 documentation - GitHub Pages](#)
-

User prompt: Nom? Handrolling only the couple combinators winnow needs? Memchr style pre scan for new lines to par iter parse at chapter boundaries since chapter boundaries are hard boundaries in one sense (with a few complexities such as milestones?)

Response: 1. What about Nom?

Winnow is explicitly the modern rewrite of Nom (v7+).

Both were maintained by the same core author (Ed Page), but Winnow was created to fix fundamental ergonomic and performance issues in Nom:

- **Mutable Slice Parsing:** Nom passes immutable slices around, forcing combinators to return `Result<(Input, Output), Error>`. Winnow passes a mutable reference (`&mut Input`), which drastically cuts down on slice copying/pointer updates and makes Rust borrow checker interactions far cleaner.
- **Better Diagnostics:** Winnow's error reporting and tracking are much better suited for language/markup tooling.

If you are choosing between the two today, **Winnow is strictly superior to Nom** for a new Rust project.

2. Hand-rolling only the combinators you need?

This is actually a very common and highly effective pattern in Rust for markup parsing.

If your grammar doesn't need complex monadic combinators or generalized abstract trait bounds, writing custom hand-rolled helper functions on top of raw `&str` / `&[u8]` methods is often **faster to compile, easier to debug, and just as fast at runtime**.

A "combinator" in plain Rust can literally just be a function with this signature:

```
type Span<'a> = &'a str;
type ParseResult<'a, T> = Option<(T, Span<'a>)>;

// A hand-rolled "take while"
inline fn take_while<'a, F>(input: Span<'a>, mut predicate: F) -> (Span<'a>, Span<'a>)
where F: FnMut(char) -> bool {
    let offset = input.find(|c| !predicate(c)).unwrap_or(input.len());
    (&input[..offset], &input[offset..])
}
```

Pros of Hand-rolling:

- **Zero Macro/Trait Generic Bloat:** Nom/Winnow heavily utilize trait-heavy generics that can bloat compile times.
- **Direct Control Over State:** USFM requires tracking state (e.g., *Are we inside a footnote? What was the last open character marker?*). Passing a simple `struct ParserState<'a>` directly into hand-rolled functions avoids jumping through combinator hooks.

Cons:

- You have to implement slice bounds tracking, line/column tracking, and error backtracks manually.

3. Parallel Parsing via `memchr` Pre-scanning on Chapter Boundaries (`\c`)

This strategy is solid, provided you account for USFM's specific structural quirks.

USFM files (especially whole Bibles) are large text files. Because chapters (`\c 1`) are high-level structural divisions, scanning for `\c` boundaries and dispatching them across threads via `rayon` is a great way to achieve multi-gigabyte-per-second throughput.

Original USFM File:

```
[ Header / Id ] ----> Rayon Thread 0
[ \c 1 ...    ] ----> Rayon Thread 1
[ \c 2 ...    ] ----> Rayon Thread 2
```

How to implement the pre-scan safely:

1. **Use `memchr` or `memmem` to find `\n\c` or `\r\n\c` :** You cannot *just* search for `\c` anywhere, because `\c` could appear in verse text, inside an attribute value, or as part of another marker like `\c ls` or `\cp`. Searching specifically for byte sequences matching **newline + `\c` + space** (e.g., `b"\n\c "` or `b"\r\n\c "`) gives you true chapter boundaries with nearly 100% reliability.
2. **The Milestone Complexity (`\qt-s*`, `\ms*`):** You mentioned USFM milestones. Milestones in USFM 3.0 (like `\qt-s*` ... `\qt-e*` for quotation spans) can start in Chapter 1 and end in Chapter 3.
 - **The Lexing/AST Phase is Independent:** As long as your parallel chapter parser outputs a stream of **AST nodes / Tokens** per chapter, you can safely parse the text of Chapter 1, Chapter 2, and Chapter 3 in parallel.
 - **The Resolution Phase is Sequential:** A lightweight second pass (or stream fold) stitches the ASTs together to resolve cross-chapter milestone pairs or validate unclosed attributes.

Recommended Architecture for USFM

If you want maximum performance and minimal dependencies:

1. **Pre-scan Stage (`memchr`):** Scan the byte buffer for `\n\c` boundaries. Split the raw `&str` buffer into chapter slices.
 2. **Parallel Lex/Parse Stage (`rayon`):** Hand-roll a simple `&str` slicing parser (or use `winnow`) to parse each chapter slice independently into a `ChapterAST` struct.
 3. **Stitch / Validate Stage:** Run a fast, sequential pass over the array of `ChapterAST`s to track global document state, resolve milestones, and output your final data structure.
-

User prompt: Oh, Chomsky?

Response: Spot on. You've hit the exact core of formal language theory!

What you're running into with USFM is the classic boundary between **Type-3 (Regular)** and **Type-2 (Context-Free)** languages on the **Chomsky Hierarchy**:

The Chomsky Hierarchy Breakdown

```
[ Type-0: Unrestricted / Turing ]
      ▲
[ Type-1: Context-Sensitive ]
      ▲
[ Type-2: Context-Free ] <--- (Winnow / Hand-rolled Parser)
      ▲
[ Type-3: Regular ] <--- (Logos / DFA)
```

1. Type-3 (Regular Languages / Logos):

- **Power:** Finite State Automata (DFA/NFA).
- **Rule:** Memoryless. The engine only knows its current state and the incoming byte. It has no concept of stack frames, recursion, or balanced nesting.

- **Why it fails on USFM:** Logos is a Type-3 engine. It can't easily handle state changes like "switch parsing rules because we crossed a | character inside an open \w marker," or track nested scopes (\f inside \v).

2. Type-2 (Context-Free Languages / Winnow or Hand-rolled):

- **Power:** Pushdown Automata (Engine with a memory stack or call stack).
- **Rule:** Can handle recursion, nested structures, and contextual switching by maintaining a stack of grammar rules.
- **Why USFM needs this:** Even though USFM markup looks line-oriented, nested character styles, footnotes (\f ... \f*), attributes, and milestone spans push it firmly into Type-2 territory.

Why USFM Trashes "Pure" Chomsky Classifications

To make matters worse, USFM technically touches **Type-1 (Context-Sensitive)** behavior in practice:

- **Milestones (\qt1-s* ... \qt1-e*):** A milestone closing tag must match its specific opening tag type across arbitrary distances (and across chapter boundaries).
- **Delimiters:** The meaning of a space character changes dynamically based on the exact token immediately preceding it (e.g., \v 12 vs verse text).

By skipping a strict Type-3 DFA (Logos) and using **Winnow or hand-rolled slice parsing**, you jump up the Chomsky hierarchy to give your code the stack and state context it needs to digest USFM smoothly.

User prompt: No I meant the rust library vs winnow.

Response: Ah, **chomsky** the Rust crate! My mistake—I assumed we were talking theoretical computer science!

When comparing the **chomsky crate** versus **winnow**, you are looking at two fundamentally different design philosophies for parser combinators in Rust: **Pure Combinator Algebra** vs. **Pragmatic In-Place Slice Mutation**.

The Fundamental Difference

Feature	<code>winnow</code>	<code>chomsky</code>
Philosophy	Pragmatic & Fast. Idiomatic, imperative-feeling slice mutation.	Pure & Functional. Mathematical combinator composition.
Input Model	<code>&mut Input</code> (in-place slice mutation).	Takes immutable input, returns <code>(Output, RemainingInput)</code> .
Memory / Allocations	Zero-copy slicing, minimal stack depth.	Zero-copy, but heavier on generic function wrappers.
Compile Times	Fast to Moderate (monomorphization is well-optimized).	Slower (deep nested generic trait bounds).
Ecosystem & Usage	Heavy adoption (Cargo team, standard tools).	Niche / Experimental functional crate.

1. The Architectural Split

How `winnow` Works

`winnow` operates on a mutable reference to your input (`&mut &str` or `&mut &[u8]`). When a combinator runs, it advances the pointer directly in memory.

- **Why it matters for USFM:** As you process USFM markers, text, and attributes, you are simply shifting a slice boundary forward. There is no tuple shuffling or complex stack unwinding.

How `chomsky` Works

`chomsky` follows the classic Haskell / functional parser combinator model (similar to early `nom` or `chumsky`—though worth noting if you meant **chumsky**, see the note below!). It builds parsers by composing deeply nested types and returning tuples of `Result<(Output, Remaining), Error>`.

2. Did you mean `chumsky` (with a 'u')?

If you meant **chumsky** (the extremely popular parser combinator crate known for its friendly error recovery):

`chumsky` is famous for **error recovery** (e.g., if a user forgets a closing `\f*` in USFM, `chumsky` can automatically recover, report a clean error with spans, and keep parsing the rest of the file).

However, **`chumsky` is significantly slower than `winnow`** for high-throughput text processing:

- **`winnow`** prioritizes raw performance (GB/s parsing speeds) and low compile times.
- **`chumsky`** prioritizes expressive error diagnostics, IDE support, and AST recovery at the expense of compile time and runtime throughput.

Summary for USFM

If you want to parse USFM Bibles quickly (especially with your `memchr` parallel chapter scanning strategy):

1. **`winnow` (or Hand-rolled)**: The right tool for max throughput, low compile overhead, and simple slice manipulation.
 2. **`chumsky`**: Only if your primary goal is building a USFM Linter / Language Server (LSP) where reporting every single typo or unclosed milestone with precise span annotations is more important than raw speed.
 3. **`chomsky` (with an 'o')**: Avoid for this use case—it lacks the ecosystem maturity, performance tuning, and ergonomic slice mutation of `winnow`.
-

User prompt: What's memmem?

Response: You likely mean `memmem`, which is the substring equivalent of `memchr`.

Where standard `memchr` searches a byte buffer for a **single byte** (e.g., finding the next `\n`), `memmem` searches a byte buffer for a **multi-byte sequence / substring** (e.g., finding the next `\n\c` or `\r\n\c`).

In the Rust ecosystem, `memmem` is provided as a module inside the widely used [memchr crate](#) by Andrew Gallant (burntsushi).

Why `memmem` Is Unreasonably Fast

Under the hood, `memmem` doesn't use naive $O(N \times M)$ string scanning. Instead, it uses heavily optimized vector SIMD algorithms (specifically variation strategies based on **Teddy** and **Two-Way** search algorithms).

When you run `memmem::find(haystack, b"\n\\c ")`:

1. **SIMD Vectorization**: It loads 16, 32, or 64 bytes at a time into AVX2/AVX-512/NEON registers.
2. **Rare-Byte Hoisting**: It identifies the rarest byte in your needle (e.g., `\` in `\n\\c`) and uses vector instructions to scan gigabytes of memory per second for *just* that byte candidate before verifying the full match.
3. **Branchless**: It bypasses standard byte-by-byte loops entirely.

How You Use It for USFM Parallelization

If you want to split a massive USFM Bible file into chapters across `rayon` threads, `memchr::memmem` is the exact tool to find the chunk boundaries instantly:

```
use memchr::memmem;

fn find_chapter_boundaries<'a>(usfm_bytes: &'a [u8]) -> Vec<usize> {
    // Search specifically for newline + backslash + c + space
    let finder = memmem::Finder::new(b"\n\\c ");

    // Returns byte offsets where every new chapter starts in the raw buffer
    finder.find_iter(usfm_bytes).collect()
}
```

Because `memmem` runs at near-hardware-memory-bandwidth speeds (10–30+ GB/s depending on CPU), pre-scanning a full 5MB Bible text file to extract all 1,189 chapter slice boundaries takes **microseconds** before sending them off to parallel worker threads.

User prompt: So it's not that you think logos is faster as much as it's gonna require a post process parse that could be done via one lex and parse? Cause somehow it seems like it's compiled jump tables beat 1x1 byte scanning?

Response: Exactly. It isn't that Logos isn't fast at moving its pointer across bytes—it's that **a pure DFA lexer alone cannot express USFM's grammar without inventing artificial tokens and pushing the actual structural logic into a second pass.**

When you weigh Logos vs. Winnow (or a hand-rolled parser), the performance equation isn't just `DFA vs. 1x1 Byte Loop`. It comes down to **Architectural Efficiency**:

1. Logos DFA vs. SIMD / `memchr` Byte Scanning

It's a common misconception that a hand-rolled or Winnow-based parser scans bytes strictly one by one in a naive `for` loop.

When you write Winnow or custom `&str` parsing for USFM, you aren't inspecting every character individually. You use slice operations like `take_till(|c| c == '\\\ ' || c == '|')` or `memchr`, which under the hood delegate to **SIMD instructions** that sweep through 16 to 64 bytes per CPU cycle.

- **Logos (DFA Jump Table):** Inspects bytes and jumps through state tables. It excels at complex token character classes, but it still evaluates character-by-character along its state transitions.
- **Winnow / `memchr` Slice Sweeps:** For large spans of plain USFM verse text (which makes up ~80% of a Bible file), `memchr` scans for the next `\` or `|` at 10–20 GB/s using SIMD hardware vector registers. A DFA table jump cannot match vector register throughput over plain text spans.

2. The "Two-Pass" Tax

If you use Logos for USFM, your pipeline looks like this:

```
[Raw USFM] —(Pass 1: Logos DFA)—> [Vec<Token<'a>>] —(Pass 2: AST Parser)—> [Document AST]
```

To make USFM work in Logos without state errors, Logos has to emit a stream of raw, un-contextualized tokens (e.g., `Backslash`, `Word("v")`, `Whitespace`, `Digits("12")`, `Text(...)`).

1. **Memory / Allocations:** Even if tokens are zero-copy references (`&'a str`), collecting millions of token enums into a `Vec` or traversing an iterator stream incurs overhead in CPU cache allocation and memory bandwidth.
2. **Duplicated Traversal:** Pass 1 inspects every byte to make tokens. Pass 2 inspects every token to build structures (like binding verse text to chapter markers or grouping footnote bodies).

3. The Single-Pass Winnow / Hand-Rolled Alternative

By integrating parsing and slicing into a single pass:

```
[Raw USFM] —(Single Pass: SIMD Slice Sweeps)—> [Document AST]
```

- When you are in **verse text mode**, you don't run a state machine on every word. You do a single hyper-fast `memchr` vector scan directly to the next `\` or `|` and capture that entire 200-byte sentence as a single slice instantly.

- When you hit `\`, you read the marker string (`v`, `p`, `f`), make your structural AST decision **immediately on the stack**, and jump right back to vector scanning text.

Summary

Logos has the faster **general-purpose token state machine**. But for USFM markup, **you don't actually want a general-purpose token state machine for every byte of verse text**.

Combining `memchr` vector sweeps over raw text spans with immediate structural decisions in a single pass will virtually always outperform tokenizing everything through a DFA first and running a second AST pass on the output.

User prompt: How would you combine `winnow` with `memchr` or `memchr`?

Response: Combining `winnow` with `memchr` / `memchr` is straightforward because `winnow` is built to work natively on `&str` and `&[u8]` slices.

Instead of writing a `winnow` combinator that tests bytes individually inside a predicate closure, you use `memchr` or `memchr` to **instantly find the byte index of your boundary**, split the slice at that index, and advance `winnow`'s internal pointer.

Here is how you wire them together cleanly.

1. Extending `winnow` with Custom `memchr` Parsers

In `winnow`, a parser is just a function that takes `&mut Input` and returns a `PResult<Output>`.

You can write custom, reusable helper functions (or extension traits) that invoke `memchr` directly on the input slice:

```
use winnow::prelude::*;
use winnow::error::ContextError;
use memchr::memchr;

/// Parses plain text up to the next occurrence of `delimiter` byte (e.g., `\" or `|`).
/// Uses SIMD-accelerated `memchr` under the hood.
pub fn take_until_byte<'s>(
    delimiter: u8
) -> impl FnMut(&mut &'s str) -> PResult<&'s str, ContextError> {
```

```

move |input: &mut &'s str| {
    // Access raw bytes without copying
    let bytes = input.as_bytes();

    match memchr(delimiter, bytes) {
        Some(idx) => {
            // Split the string slice at the SIMD-located boundary
            let (head, tail) = input.split_at(idx);
            *input = tail; // Advance winnow's input reference
            Ok(head)
        }
        None => {
            // If delimiter isn't found, consume the remaining slice entirely
            let head = *input;
            *input = "";
            Ok(head)
        }
    }
}
}
}

```

2. Multi-Byte Sequences with `memmem`

For USFM structural boundaries like end-markers (`\f*`, `\+w*`) or chapter markers (`\c`), you use `memmem::Finder`:

```

use memchr::memmem::Finder;
use winnow::prelude::*;
use winnow::error::ContextError;

/// Takes text up to a multi-byte sequence like `\f*` using SIMD `memmem`.
pub fn take_until_sequence<'s>(
    finder: &'static Finder<'static>
) -> impl FnMut(&mut &'s str) -> PResult<&'s str, ContextError> {
    move |input: &mut &'s str| {
        let bytes = input.as_bytes();

        match finder.find(bytes) {

```

```

        Some(idx) => {
            let (head, tail) = input.split_at(idx);
            *input = tail;
            Ok(head)
        }
        None => {
            let head = *input;
            *input = "";
            Ok(head)
        }
    }
}
}
}

```

3. Putting It Together: A Single-Pass USFM Fragment Parser

Now you can mix standard `winnow` combinators (for small, precise tokens like `\v 12`) with `memchr` / `memmem` (for sweeping over hundreds of bytes of verse text at 10+ GB/s):

```

use winnow::prelude::*;
use winnow::token::take_while;
use winnow::ascii::digit1;
use memchr::memchr2;
use std::sync::LazyLock;
use memchr::memmem::Finder;

static FOOTNOTE_END: LazyLock<Finder<'static>> = LazyLock::new(|| Finder::new(b"\\f*"));

#[derive(Debug, PartialEq)]
pub enum UsfmNode<'a> {
    VerseNumber(&'a str),
    Text(&'a str),
    Footnote(&'a str),
}

// 1. Precise Winnow parser for \v 12
fn parse_verse_marker<'s>(input: &mut &'s str) -> PResult<&'s str> {

```

```

    "\\v ".parse_next(input)?;
    let num = digit1.parse_next(input)?;
    let _ = " ".parse_next(input); // Optional trailing delimiter space
    Ok(num)
}

// 2. SIMD-accelerated text sweep up to the next \ or |
fn parse_plain_text<'s>(input: &mut &'s str) -> PResult<&'s str> {
    let bytes = input.as_bytes();

    // memchr2 searches for either '\\\ OR '|' simultaneously in SIMD registers
    match memchr2(b'\\', b'|', bytes) {
        Some(0) => Err(winnow::error::ErrMode::Backtrack(
            winnow::error::ContextError::default(),
        )),
        Some(idx) => {
            let (head, tail) = input.split_at(idx);
            *input = tail;
            Ok(head)
        }
        None if !input.is_empty() => {
            let head = *input;
            *input = "";
            Ok(head)
        }
        None => Err(winnow::error::ErrMode::Backtrack(
            winnow::error::ContextError::default(),
        )),
    }
}

// 3. Fast Footnote parser switching to memmem finder
fn parse_footnote<'s>(input: &mut &'s str) -> PResult<&'s str> {
    "\\f ".parse_next(input)?;
    let content = take_until_sequence(&FOOTNOTE_END).parse_next(input)?;
    "\\f*".parse_next(input)?;
    Ok(content)
}

```

```

}

// Master parser dispatch
pub fn parse_usfm_node<'s>(input: &mut &'s str) -> PResult<UsfmNode<'s>> {
    alt((
        parse_verse_marker.map(UsfmNode::VerseNumber),
        parse_footnote.map(UsfmNode::Footnote),
        parse_plain_text.map(UsfmNode::Text),
    )).parse_next(input)
}

```

Why This Combination Is so Effective

1. **Zero Allocations:** Everything remains a `&'a str` slice of the original memory buffer.
 2. **Optimal Branching:** `winnow` handles structural matching and error backtracking near markers, while `memchr` skips whole blocks of text without running character-by-character loops.
 3. **No Intermediate Token Buffer:** You generate your AST nodes (e.g. `UsfmNode::Text`) on the fly in a single pass.
-

User prompt: You said threading parser state through with winnow was finicky though?

Response: Ah, that's my fault for not being clearer earlier—let's untangle that!

Threading state in `winnow` isn't *impossible*, but if you try to do it using `winnow`'s built-in state mechanism (`Stateful<I, S>`), it can feel **finicky and type-heavy** compared to simple imperative Rust.

Here is why it gets awkward, and how to cleanly bypass the finickiness if you need parser state.

Why `winnow`'s Built-In `Stateful` Feels Finicky

`winnow` provides a wrapper called `Stateful<Input, State>`. The friction comes from two places:

1. **Borrow Checker Fights inside Closures:**

Because `winnow` combinators take `&mut Input`, when `Input` is a `Stateful<&str, MyState>`, accessing both the string slice and updating your state at the same time requires splitting borrows on the wrapper struct. In deeply nested combinator closures (`alt`, `dispatch`, `repeat`), Rust's borrow checker will frequently complain about partial moves or mutable references overlapping.

2. Monadic Type Pollution:

As soon as a helper parser expects a `Stateful` input, its function signature changes. You can no longer easily compose a plain `fn(&mut &str) -> PResult<&str>` with a stateful parser without constantly wrapping and unwrapping the state context.

How to Keep State Clean (3 Approaches)

Depending on what kind of state you actually need for USFM, here is how you avoid the friction:

Approach 1: The "Context-Free First Pass" (Recommended for USFM)

For 90% of USFM parsing, **you don't actually need mutable parser state during the slice phase.**

Instead, construct a clean, stateless AST or token stream in pass 1, and then resolve contextual state (like matching milestone opening/closing pairs or tracking unclosed character markers) in a fast, sequential pass over the AST.

```
// Pass 1: Stateless Winnow + memchr pass -> UsfmNode
// Pass 2: Simple imperative loop to track context
struct UsfmContext {
    current_chapter: u32,
    open_markers: Vec<String>,
}

fn validate_and_resolve(nodes: Vec<UsfmNode>) {
    let mut ctx = UsfmContext::default();
    for node in nodes {
        match node {
            UsfmNode::Marker("c") => { /* update ctx */,
            UsfmNode::MilestoneStart(id) => { ctx.open_markers.push(id); },
            // ...
        }
    }
}
```

```
}
}
```

Approach 2: Method-Based Struct Parser (Zero Combinator Finickiness)

Instead of using `winnow`'s monadic `Stateful` wrapper or passing `&mut Context` through pure combinators, wrap the input slice AND state inside your own Parser struct, and write standard Rust methods:

```
pub struct UsfmParser<'a> {
    pub input: &'a str,
    pub current_chapter: u32,
    pub in_footnote: bool,
}

impl<'a> UsfmParser<'a> {
    // A method that uses winnow internally, but has direct &mut self access!
    pub fn parse_next_node(&mut self) -> PResult<UsfmNode<'a>> {
        if self.in_footnote {
            self.parse_footnote_content()
        } else {
            // Mutate self.input via winnow while freely reading/mutating self.current_chapter
            parse_verse_marker.parse_next(&mut self.input)?;
            Ok(UsfmNode::....)
        }
    }
}
```

Why this works: You get full `&mut self` access to your custom state fields without fighting closures, generic trait bounds, or wrapper types.

Approach 3: Threading Explicit `&mut State` as a Parameter

If you *do* want standalone functions, skip `winnow::Stateful` and just pass your state explicitly alongside the input reference:

```
fn parse_marker<'s>(
    input: &mut &'s str,
    state: &mut ParserState
```

```
) -> PResult<UsfmNode<'s>> {  
    let name = parse_marker_name.parse_next(input)?;  
    state.last_marker = Some(name); // Clean, standard Rust mutation  
    Ok(UsfmNode::Marker(name))  
}
```

Bottom Line

The "finicky" reputation comes from trying to force `winnow` to manage state through its generic functional combinators.

If you use **Winnow purely for slice advancement** (`&mut &str`) and use standard **Rust struct methods or explicit `&mut State` args** for state, it stays fast, ergonomic, and completely painless.
